Chapter 18 — Learning Roadmap

A 24-week plan (about 10 hours/week). Compress by skipping labs for topics you already know; every phase ends with a mastery checkpoint you must pass before moving on.

18.1 Phase 1 — Foundations (Weeks 1–4)

Conceptual Deep Dive

Chapters 2–5. Goal: fluent Python for async I/O, reliable API clients, and a working mental model of transformers, tokens and cost.

Milestone: build a resilient async LLM client.

Labs 1. Implement a token-bucket limiter + retry with jitter + circuit breaker; load-test against a mock server that returns random 429/503. 2. Train a toy BPE tokenizer and compare token counts to a real tokenizer across English, code and another language. 3. Implement causal self-attention from scratch and verify against `torch.nn.functional.scaled_dot_product_attention`. 4. Compute KV-cache and weight memory for three open models.

Mastery checkpoint: explain, without notes, why decode is memory-bound, how retries can cause outages, and what a KV cache stores.

Cross-Connection Scenarios

These labs reappear inside serving (13) and evaluation harnesses (10).

Production Anti-Patterns

- Reading without building.
- Skipping the math labs and hitting a wall at fine-tuning.

Minimal Working Code

```
import torch
import torch.nn.functional as F

def lab_check(q, k, v):
    mine = (q @ k.transpose(-2, -1) / q.shape[-1] ** 0.5).softmax(-1) @ v
    ref = F.scaled_dot_product_attention(q, k, v)
    return torch.allclose(mine, ref, atol=1e-5)

x = torch.randn(1, 2, 4, 8)
print(lab_check(x, x, x))
```

18.2 Phase 2 — Building with Models (Weeks 5–10)

Conceptual Deep Dive

Chapters 6–9. Goal: prompts, structured output, tools, embeddings and a RAG system with measured quality.

Milestone: a RAG app over a corpus you choose (e.g. a project's docs) with a 100-question golden set.

Labs 1. Build a prompt registry; A/B two prompt versions on 50 cases. 2. Extract structured data from messy PDFs with Pydantic validation and a repair loop; report the repair rate. 3. Implement hybrid retrieval (BM25 + dense + RRF); compare recall@5 vs either alone. 4. Add a reranker; plot latency vs nDCG. 5. Try three chunking strategies; choose with data.

Mastery checkpoint: show a table proving which technique helped, by how much, and at what latency cost.

Cross-Connection Scenarios

Phase 3 turns these measurements into gates.

Production Anti-Patterns

- Tuning on the dev set until it is memorised.
- Adopting GraphRAG before a hybrid baseline.

Minimal Working Code

```
def compare(configs: dict, dataset, run, metric):
    return {name: sum(metric(run(cfg, ex), ex) for ex in dataset) / len(dataset)
            for name, cfg in configs.items()}
```

18.3 Phase 3 — Quality, Agents and Safety (Weeks 11–16)

Conceptual Deep Dive

Chapters 10, 11, 14. Goal: rigorous evaluation, an agent with tools, and defence in depth.

Milestone: agent with an automated eval harness and red-team report.

Labs 1. Calibrate an LLM judge against 50 human labels; report agreement (κ). 2. Build a LangGraph agent with checkpointing and human approval; measure pass^3. 3. Add tracing (OTel + Phoenix or Langfuse); find the slowest span; fix it. 4. Red-team your app with 30 injection attempts; implement three mitigations; re-test. 5. Add PII redaction and measure recall on a labelled sample.

Mastery checkpoint: present a postmortem of a failure you found, including the regression test you added.

Cross-Connection Scenarios

Phase 4 operationalises these.

Production Anti-Patterns

- Agents without budgets.
- Safety work postponed until "after launch".

Minimal Working Code

```
def cohens_kappa(a: list[int], b: list[int]) -> float:
    n = len(a)
    po = sum(x == y for x, y in zip(a, b)) / n
    labels = set(a) | set(b)
    pe = sum((a.count(l) / n) * (b.count(l) / n) for l in labels)
    return (po - pe) / (1 - pe) if pe != 1 else 1.0

print(cohens_kappa([1, 1, 0, 0, 1], [1, 0, 0, 0, 1]))
```

18.4 Phase 4 — Customisation and Operations (Weeks 17–22)

Conceptual Deep Dive

Chapters 12, 13, 15. Goal: fine-tune, serve, and release with CI gates.

Milestone: fine-tuned small model served on vLLM behind a gateway with a CI eval gate and canary.

Labs 1. QLoRA fine-tune a 1–3B model on 1k curated examples; compare to the prompt baseline on a frozen test set. 2. Quantise to AWQ/GGUF; measure quality, memory and throughput. 3. Serve with vLLM; load-test; tune max_model_len and batch settings. 4. Containerise, deploy to Kubernetes (kind/minikube or cloud), add KEDA autoscaling on queue depth. 5. Write a GitHub Actions workflow with an eval gate and rollback criteria.

Mastery checkpoint: show cost per successful task for API vs self-hosted at three traffic levels.

Cross-Connection Scenarios

Capstone integrates everything.

Production Anti-Patterns

- Fine-tuning before establishing a prompt baseline.
- Deploying without load tests.

Minimal Working Code

```
def breakeven_volume(api_cost_per_req, fixed_monthly, selfhost_cost_pe
r_req):
    return fixed_monthly / max(api_cost_per_req - selfhost_cost_per_req, 1e-9)

print(int(breakeven_volume(0.004, 3000, 0.0005)))
```

18.5 Phase 5 — Capstone and Career (Weeks 23–24)

Conceptual Deep Dive

Chapters 16, 17. Choose one project from Chapter 17, finish the readiness checklist, publish a write-up (architecture, eval results, costs, failures), and rehearse three system-design interviews. Ongoing habits: read one paper or release note per week, re-run your eval suite when a model updates, contribute to an open-source project (vLLM, LangGraph, Ragas, Phoenix, Qdrant docs), and keep a "failure journal".

Cross-Connection Scenarios

Open-source projects to study: vLLM, llama.cpp, Unsloth, TRL, PEFT, LangGraph, DSPy, Instructor, Outlines, XGrammar, Ragas, Phoenix, Langfuse, Qdrant, Presidio, NeMo Guardrails, promptfoo.

Production Anti-Patterns

- Ending with a private repo and no write-up.
- Learning only tools, not trade-offs.

Minimal Working Code

```
MILESTONES = ["resilient client", "rag + golden set&quo
t;, "agent + red team", "finetune + serve", "capstone write-up"]

def progress(done: set[str]) -> str:
    return f"{len([m for m in MILESTONES if m in done])}/{len(MILESTONES)} milestones"

print(progress({"resilient client", "rag + golden set"}))
```

18.6 Weekly Study Cadence and Retention Techniques

Conceptual Deep Dive

A sustainable weekly loop: **two focused build sessions** (2–3 hours each) on the current lab, **one reading session** (a chapter section plus one primary source such as a paper or documentation page), **one review session** (re-derive a formula or re-implement a snippet from memory, update your failure journal), and **one short write-up** (a README or blog-style note). Use spaced repetition for facts that matter in interviews (formulas, trade-offs) and "teach-back" (explain it aloud or in writing without notes). Keep a **decision log**: what you tried, the measured result, and why you chose. Edge cases: tutorial hell (consuming without building), perfectionism blocking shipping, and burnout from chasing every release. Time-box exploration of new tools to two hours before deciding whether they earn a place in your stack.

Cross-Connection Scenarios

Mastery checkpoints (18.1–18.5), portfolio (16.4), interview preparation (16.3).

Production Anti-Patterns

- Watching courses without shipping a lab.
- Skipping the review session, so earlier phases decay.
- Adopting every new framework instead of deepening fundamentals.

Minimal Working Code

```
from datetime import date, timedelta

def next_review(interval_days: int, correct: bool, ease: float = 2.0) -> tuple[int, float]:
    """Tiny spaced-repetition scheduler (SM-2-like)."""
    if not correct:
        return 1, max(1.3, ease - 0.2)
    return max(1, round(interval_days * ease)), ease + 0.05

interval, ease = 1, 2.0
for ok in [True, True, False, True]:
    interval, ease = next_review(interval, ok, ease)
print(date(2026, 10, 22) + timedelta(days=interval), round(ease, 2))
```

```
import json
from pathlib import Path

def log_decision(path: str, what: str, result: str, decision: str):
    p = Path(path)
    items = json.loads(p.read_text()) if p.exists() else []
    items.append({"what": what, "result": result, "decision": decision})
    p.write_text(json.dumps(items, indent=2))

log_decision("decisions.json", "chunk 400 vs 800 tokens", "recall@5 .81 vs .78&q
uot;, "use 400")
```

18.7 Portfolio Projects, Open-Source Contribution and Specialisation Tracks

Conceptual Deep Dive

A strong portfolio has **three** shipped projects with: a README (problem, architecture diagram, how to run), a measured evaluation (dataset, metrics, intervals), a cost and latency analysis, a failure-analysis section, and a deployed demo or video. Suggested set: a RAG system with eval, an agent or tool-using workflow with safety controls, and a fine-tuning-and-serving or platform project. **Open-source contribution ladder**: improve docs and examples, add tests, fix small bugs, implement a feature, then maintain a component; start from "good first issue" labels in projects you actually use. **Specialisation tracks** after the core path: *applied/product* (UX, evals, rapid iteration), *platform/infra* (serving, GPUs, reliability), *evaluation & safety* (red-teaming, judges, governance), *model customisation* (fine-tuning, distillation, preference optimisation), *retrieval/search* (embeddings, ranking,

indexing). Choose one for depth while keeping working breadth. Edge cases: portfolios built on tutorial datasets, demos that break without your API key, and unlicensed code.

Cross-Connection Scenarios

Capstone checklist (17.6), career ladder (16.4), project blueprints (17).

Production Anti-Patterns

- Notebook-only projects with no evaluation.
- Demo repos requiring undocumented secrets.
- Contributing nothing back to tools you depend on.

Minimal Working Code

```
REQUIRED = ["README", "architecture_diagram", &quo
t;eval_results_with_ci", "cost_latency_analysis",
           "failure_analysis", "run_instructions", "license"]

def portfolio_review(project: dict) -> dict:
    missing = [r for r in REQUIRED if not project.get(r)]
    return {"name": project["name"], "score": 1 - len(missing) / len(REQUIR
ED), "missing": missing}

print(portfolio_review({"name": "rag-assistant", "README": True, "eval
_results_with_ci": True}))
```

```
TRACKS = {
    "applied": {"evals", "prompting", "product_sense"},
    "platform": {"serving", "kubernetes", "reliability"},
    "eval_safety": {"red_team", "judges", "governance"},
    "customisation": {"lora", "dpo", "distillation"},
    "retrieval": {"embeddings", "ranking", "indexing"},
}

def best_track(skills: set[str]) -> str:
    return max(TRACKS, key=lambda t: len(TRACKS[t] & skills))

print(best_track({"lora", "dpo", "evals"}))
```

18.8 Annotated Reading List: Foundational Papers and Sources

Conceptual Deep Dive

Read primary sources in this order, spending 1–2 hours each: skim the abstract and figures, read the method, then the limitations. (Titles are well-known; verify details on arXiv.) **Architecture**: *Attention Is All You Need* (Vaswani et al., 2017); *BERT* (Devlin et al., 2018); *RoFormer / RoPE* (Su et al., 2021); *FlashAttention* (Dao et al., 2022); *GQA* (Ainslie et al., 2023); *Mamba* (Gu and Dao, 2023). **Scaling and training**: *Scaling Laws for Neural Language Models* (Kaplan et al., 2020); *Training Compute-Optimal Large Language Models* ("Chinchilla", Hoffmann et al., 2022); *Training language models to follow instructions with human feedback* ("InstructGPT", Ouyang et al., 2022); *Constitutional AI* (Bai et al., 2022). **Prompting and reasoning**: *Chain-of-Thought Prompting* (Wei et al., 2022); *Self-Consistency* (Wang et al., 2022); *ReAct* (Yao et al., 2022); *Tree of Thoughts* (Yao et al., 2023); *Toolformer* (Schick et al., 2023); *Lost in the Middle* (Liu et al., 2023). **Retrieval**: *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (Lewis et al., 2020); *ColBERT* (Khattab and Zaharia, 2020); *HyDE* (Gao et al., 2022); *Self-RAG* (Asai et al., 2023); *RAPTOR* (Sarthi et al., 2024); *From Local to Global: A GraphRAG Approach* (Edge et al., 2024); *Matryoshka Representation Learning* (Kusupati et al., 2022). **Efficient adaptation and serving**: *LoRA* (Hu et al., 2021); *QLoRA* (Dettmers et al., 2023); *GPTQ* (Frantar et al., 2022); *AWQ* (Lin et al., 2023); *Direct Preference Optimization* (Rafailov et al., 2023); *Efficient Memory Management for LLM Serving with PagedAttention* (Kwon et al., 2023); *Fast Inference via Speculative Decoding* (Leviathan et al., 2022); *DeepSeekMath* (introduces GRPO; Shao et al., 2024). **Agents and evaluation**: *Generative Agents* (Park et al., 2023); *Reflexion* (Shinn et al., 2023); *MemGPT* (Packer et al., 2023); *Judging LLM-as-a-Judge with MT-Bench* (Zheng et al., 2023); *RAGAS* (Es et al., 2023); *DSPy* (Khattab et al., 2023). Edge cases: papers are snapshots; check later errata, follow-ups and whether results replicate.

Cross-Connection Scenarios

Each paper maps to a chapter section (attention 5.4, RoPE 5.9, RAG 9, LoRA 12.3, DPO 12.4, PagedAttention 13.1).

Production Anti-Patterns

- Implementing from a blog summary without reading the paper's limitations.
- Reading only new papers, never the foundations.
- Taking benchmark claims at face value without replication.

Minimal Working Code

```
READING = [
    ("Attention Is All You Need", 2017, "5.4"), ("LoRA", 2021, "12.3&q
uot;), ("Chinchilla", 2022, "5.5"),
    ("Chain-of-Thought Prompting", 2022, "6.2"), ("ReAct", 2022, "6.3&
quot;), ("RAG (Lewis et al.)", 2020, "9.1"),
    ("DPO", 2023, "12.4"), ("PagedAttention", 2023, "13.1"), (&qu
ot;FlashAttention", 2022, "5.4"),
]

def next_paper(done: set[str], chapter_focus: str):
    return [t for t, y, sec in READING if sec.startswith(chapter_focus) and t not in done]

print(next_paper({"LoRA"}, "12"))
```

```
def paper_notes_template(title: str) -> str:
    return (f"# {title}\n- Problem:\n- Key idea (one sentence):\n- Method:\n- Evidence:\n"
            f"- Limitations / what would break it:\n- How I would test it on my data:\n")

print(paper_notes_template("LoRA").splitlines()[0])
```

18.9 Lab Rubrics, Grading Criteria and Solution Hints

Conceptual Deep Dive

Self-assess each lab with a rubric rather than "it runs". **Resilient client (Phase 1)**: passes if retries use jitter and honour Retry-After, the breaker opens and half-opens correctly, concurrency is bounded, timeouts are set, and a fault-injection test demonstrates all four. **RAG app (Phase 2)**: passes if there is a golden set of 100+ questions with labelled sources, recall@k and faithfulness reported with intervals, at least two ablations (chunking, hybrid, rerank), abstention when evidence is missing, and a documented failure analysis. **Agent and red-team (Phase 3)**: passes if budgets, loop detection and approvals exist, trajectory metrics (pass^k, cost per solved task) are reported, 30+ attack prompts have measured ASR before and after mitigations, and logs are redacted. **Fine-tune and serve (Phase 4)**: passes if the tuned model beats a strong prompt baseline on a frozen test set, general-capability regression is checked, quantised variants are re-evaluated, a load test with open-loop arrivals reports p95 TTFT, and a CI gate plus rollback are demonstrated. **Hints**: when stuck, write the eval first; inspect actual chunks and rendered prompts; reduce to one failing example; change one variable at a time; log everything with a request id. Edge cases: labs that pass on your laptop but not in a clean container; keep a Dockerfile.

Cross-Connection Scenarios

Mastery checkpoints (18.1–18.5), readiness checklist (17.6), testing (15.8).

Production Anti-Patterns

- Declaring a lab done because the demo worked once.
- No ablations, so improvements cannot be attributed.
- Skipping the failure analysis write-up.

Minimal Working Code

```
RUBRICS = {
    "resilient_client": ["jittered_backoff", "retry_after", "circuit_b
reaker", "bounded_concurrency", "fault_injection_test"],
    "rag_app": ["golden_set_100", "recall_and_faithfulness_ci", "two_a
blations", "abstention", "failure_analysis"],
    "agent_redteam": ["budgets", "loop_detection", "approvals", &
quot;pass_hat_k", "asr_before_after"],
    "finetune_serve": ["beats_prompt_baseline", "general_regression_check",
 "quant_reeval", "open_loop_load_test", "ci_gate_rollback"],
}

def grade(lab: str, achieved: set[str]) -> dict:
    items = RUBRICS[lab]
    missing = [i for i in items if i not in achieved]
    return {"lab": lab, "score": f"{len(items) - len(missing)}/{len(items)}"
;, "missing": missing}

print(grade("rag_app", {"golden_set_100", "abstention"}))
```

```
def debug_checklist(symptom: str) -> list[str]:
    base = ["write/identify a failing example", "log the rendered request and raw response
",
            "change one variable at a time", "re-run the eval, not just the example"]
    extra = {"retrieval": ["inspect the actual chunks", "check recall@50 vs reca
ll@k"],
             "latency": ["split time by stage", "check queueing vs generation&qu
ot;]}
    return base + extra.get(symptom, [])

print(debug_checklist("retrieval"))
```